

MATH 526 Lab 4 Fall 2006

Part I

In this part we learn to use the MATLAB commands **disp**, **fprintf**, **flops**, **rand** and **lu**.

The MATLAB command **disp** enables us to write a text on the screen. Try the following.

```
>>disp('This is Lab 4 of MATH526 (Fall 2006).')
```

Note that text strings should be surrounded by single quotes. Sometimes we want to display a text on the screen that will include the numerical value of a variable. The MATLAB command **fprintf** is useful for this purpose. Try the following commands.

```
>>m = 4;
>>y = 2006;
>>fprintf('This is Lab %g of MATH526 (Fall %g).\n', m, y)
```

Use **help** to learn more about **fprintf**.

The MATLAB command **flops** computes the number of floating point operations. (Use help to find out more about **flops**.) Let **A** be a 1×10 matrix of 1's, **B** be an 10×1 matrix of 1's and **C** be a 1×10 matrix of 1's. Even though matrix multiplication is associative, we will see how the placement of the parentheses affects the number of floating point operations (and hence the computing time).

```
>>A = ones(1,10);
>>B = ones(10,1);
>>C = ones(1,10);
```

Next reset the count of floating point operations to zero:

```
>>flops(0)
```

Then compute the product **(AB)C** and determine how many flops were used:

```
>>(A*B)*C
>>flops
```

Reset again, compute the product **A(BC)** and determine the number of flops:

```
>>flops(0)
>>A*(B*C)
>>flops
```

Which computation is cheaper?

Random matrices can be generated by the MATLAB command **rand**. Such matrices are useful for numerical experiments.

```
>>M = rand(4,3)
```

A random 4×3 matrix has been created. Use **help** to learn more about **rand**.

MATLAB has a built-in function that will perform the Gaussian elimination on a given matrix **A** with partial pivoting and return the corresponding **L-U-P** factorization of **A**, i.e., a unit lower triangular matrix **L**, an upper triangular matrix **U** and a permutation matrix **P** such that **PA=LU**. Create the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \\ 1 & 4 & 9 \end{bmatrix}$$

and try the following commands:

```
>>[L,U,P] = lu(A)
>>P*A
>>L*U
```

Clear all the variables before you work on Part II.